

Designing High-Throughput Event-Driven Architectures for E-Commerce Fulfillment at Global Scale

1st Karan Kumar Ratra
Senior Manager Software
Engineering
Walmart Global Tech
Sunnyvale, CA, USA
karanratra07@gmail.com

2nd Dhruv Kumar Seth
Solution Architect
Walmart Global Tech
Sunnyvale, CA, USA
er.dhruv08@gmail.com

3rd Deepika Verma
Director Software Engineering
Walmart Global Tech
Sunnyvale, CA, USA
gandhi.deepika@gmail.com

4th Hemant Burman
Director Software Engineering
Walmart Global Tech
Sunnyvale, CA, USA
hemant.burman@gmail.com

Abstract—Traditional monolithic systems struggle to scale effectively in the current global e-commerce landscape, characterized by high transaction volumes and unpredictable traffic spikes, and require real-time responsiveness. This paper conducts a systematic review of High-Throughput Event-Driven Architectures (EDAs) as the base design approach for developing flexible and responsive fulfillment systems with low latency. The research investigates core principles, including asynchronous communication, service decoupling, and event-carried state transfer, alongside advanced design patterns such as Event Sourcing, Saga orchestration, and Command Query Responsibility Segregation (CQRS). Real-world case studies from Walmart, eBay, and Shopify demonstrate how EDA can enable sub-second order processing, dynamic inventory synchronization, and rapid elasticity. The review in this paper also evaluates cloud-native event brokers, serverless frameworks, and event meshes for their ability to support global and distributed deployments at scale. Additionally, the paper includes a section specific to implementation challenges faced while implementing EDAs in real-world scenarios and the resolution of those challenges through architectural best practices, which include idempotent consumers, versioned event contracts, and Dead Letter Queues for distributed debugging, schema evolution, and event ordering. The review establishes EDA as a strategic business enabler that delivers speed, fault tolerance, and competitive agility for e-commerce fulfillment operations. Towards the end, the paper identifies three future research areas, including adaptive event routing and observability in federated systems, as well as the integration of AI-driven decision-making within event streams.

Keywords—Event-Driven Architecture (EDA), E-commerce Fulfillment, Event Sourcing, High-Throughput Systems, Event Mesh, Global Scale, Microservices, Distributed Systems, Asynchronous Communication, Event Brokers, Apache Kafka, Serverless Computing, Cloud-Native Architectures, Event Bus, Saga Pattern, Command Query Responsibility Segregation (CQRS), Scalability, Fault Tolerance, Real-time Processing, Data Consistency.

I. INTRODUCTION

In today's rapidly evolving world of global e-commerce, platforms are expected to handle large volumes of transactions

in any given second, as well as scale their fulfillment capacities to the highest degree possible. This paper thoroughly examines the importance of High-Throughput Event-Driven Architectures (EDAs) in building truly resilient and hyper-scalable e-commerce fulfillment platforms. Modern e-commerce solutions are under more pressure than ever to not only execute transactions with the lowest latency possible but also to accurately synchronize inventory in a distributed fashion in real-time and offer an optimized end-to-end experience for the consumer. Monolithic and tightly coupled synchronous architectures are no longer sufficient to meet these demands. They can ultimately lead to severe performance bottlenecks, eventual inconsistencies between various domains and/or services, and an overall inability to deliver new features at the speed the business may require. In this paper, we aim to synthesize and summarize the most relevant information, provide a deeper dive into the various architectural patterns and technologies available, and present a realistic assessment of both the advantages and challenges that exist with the adoption of EDAs. We aim to demonstrate the definitive ways in which EDAs can meet the above requirements for modern e-commerce platforms [1].

Decoupling and distributed computing have been the key differentiators that have shifted the architectural focus from monolithic to microservices-based distributed application design. An event in this scenario can be described as an immutable snapshot of state changes, or an atomic business activity or process. In the same way that microservices architecture has shifted the architectural conversation towards decoupled and event-driven inter-service communications, streaming these events asynchronously as first-class citizens in an application would lead to loose coupling and increased scalability, making an event-driven architecture the logical approach for further decoupling services via asynchronous event processing. That, in turn, would require an event streaming platform - a message broker capable of ingesting, persisting, and publishing billions of events with at least once delivery semantics and sub-millisecond latencies. This also implies leveraging Domain-Driven Design (DDD) as a design philosophy to break down the complex e-commerce domain into

smaller, bounded contexts managed by individual, autonomous services that communicate through asynchronous event streams [2].

A. The Evolving Landscape of Global E-commerce

The current e-commerce environment operates at an unprecedented level of scale, with unpredictable demand surges and significant traffic variations. The changing business environment demands systems that can scale effectively while being responsive and resilient to meet the needs of worldwide customers. The complex and dynamic requirements of modern e-commerce operations render traditional monolithic architectural approaches insufficient for addressing these needs. Such systems face significant scalability and flexibility issues when handling unpredictable, extreme loads [3], [4].

The COVID-19 pandemic revealed the fundamental limitations of these systems during a critical moment. The worldwide crisis accelerated architectural development in e-commerce by revealing major weaknesses in monolithic systems. The rapid transition to online commerce revealed that tightly connected systems lacked the necessary speed to adjust to fast-changing market demands. The pandemic-driven surge in demand created a direct relationship that led businesses to adopt microservices and Event-Driven Architecture (EDA) as essential survival mechanisms for global e-commerce success [5], [6].

B. Introduction to Event-Driven Architecture (EDA)

Event-driven architecture (EDA) is now one of the most popular and important ways to design and run distributed systems. It enables organizations to design and develop highly resilient, scalable, and responsive applications by modifying the way software components interact with one another. The basic idea is to enable asynchronous communication by producing and consuming "events" [7].

Event producers and consumers in an EDA architecture are decoupled and operate independently of each other. They react to events by processing the current state of information as it is updated across the business. The alternative to EDA is a synchronous request-response pattern, in which components often wait for each other to reply [8].

There are two main types of events in event-driven architecture: event notification and event-carried state transfer. With event notification, the event simply lets you know that something has happened but doesn't include any extra details. In that case, event consumers need to check back with the original system to get the updated state. Event-carried state transfer, on the other hand, encapsulates the current state of an entity inside an event. It means that consumers can use the data from the event to update their local replica without making an additional synchronous call [8].

The event-carried state transfer pattern has the drawback of increasing the size of events but can improve latency and resiliency as applications are less reliant on inter-service calls. This is especially relevant for high-volume e-commerce fulfillment solutions where milliseconds of latency can have a direct impact on business metrics. Events in an EDA architecture are also inherently immutable once published to the event log or event stream. This means that events can be used as

an audit trail. Event replay patterns (discussed later) can use this to provide capabilities such as fault recovery and analytics.

C. Problem Statement

The e-commerce platforms that are distributed in nature have more complex design demands in the quest to achieve international-scale high throughput. Such systems must deal with even more problems, such as the orchestration of distributed transactions with heterogeneous services, data synchronization across geographically distributed repositories that are observed by the decoupled architectures, and finally complete observability of the operations through the whole platform spectrum [9].

Traditional synchronous processing infrastructures and architectures often experience resource constraints that become the bottleneck for the expansion required by current global e-commerce workloads. These architectures are inadequate in meeting the performance standards demanded in the domain. The performance requirements associated with such workloads are largely contradictory, and one of the key obstacles is the persistence layer, or rather traditional synchronous ACID transactions, which are designed to ensure exact-once data consistency. In most of the applications, we have seen the use of ad-hoc and possibly fragile mechanisms to achieve data consistency. EDA provides a fault-tolerant solution by substituting synchronous, blocking, ACID transactions with eventual consistency generated by the execution of asynchronous events and introducing compensating transactions as a method to ensure data consistency [10].

D. Scope and Organization of the Review

This paper presents a systematic review of high-throughput event-driven systems in the e-commerce world of fulfillment, examining their application in a global context. It begins by providing an accurate explanation of the underlying concepts and theories upon which EDA is based. Then it proceeds with an expansive overview of its major strengths in the e-commerce environment. Furthermore, the paper identifies sophisticated architectural patterns and the supporting technologies that enable high-throughput and global scalability. To further illuminate these ideas, it analyzes real-life experiences of some of the most popular e-commerce websites, their practical applications, and the current reality. The paper concludes with an emphasis on the critical aspects of implementation problems inherent in the large-scale deployment of EDA, defining best practices and outlining the future course of research and development efforts in this dynamic academic and scientific area.

II. FUNDAMENTALS OF EVENT-DRIVEN ARCHITECTURE

A. Core Concepts and Principles

Event-driven architecture (EDA) is a design methodology of building applications, often using microservices, where different parts of the system communicate by sending and receiving events [11], [12]:

- **Events:** An event refers to a consequential change of state or remarkable occurrence within a system. The potential actions may be an item contained in a shopping cart or an order being shifted to the shipped status. Events

can carry complete state information, e.g., what is bought, at what price, and where it should be delivered, or they can act as an identifier, indicating that something has changed.

- **Event Producers:** The services or parts that produce and publish events are referred to as producers. The software covers frontend websites, backend microservices, Internet of Things (IoT) devices, and other Software-as-a-Service (SaaS) systems.
- **Event Consumers:** Instead, consumers subscribe to certain forms of events and take the relevant action. One service can be both a producer and a consumer at the same time: it can publish the events generated by its state transitions, and simultaneously, it can be an event processor for events generated by other services.
- **Event Brokers/Routers:** As critical middleware, event brokers take events, published by producers, and distribute them faithfully to all subscribing consumers. This central role is part and parcel of the loose coupling characteristic of EDA, in that there is no requirement for producers and consumers to have direct knowledge of each other's existence or location.

B. Key Benefits of EDA

- **Loose Coupling:** In an EDA, services are primarily independent and communicate via an event router as and when the dependency requires changes or arrangements. Independent development, deployment, and scalability of individual services are, therefore, enabled by this feature. If one of these services goes down, the decoupled architecture creates less disturbance to the other services [1].
- **Scalability & Agility:** EDA was designed to scale dynamically in response to changing workloads, making it ideal for e-commerce sites that need to account for variable and unpredictable traffic patterns. It also offers the flexibility to make changes that are easy to modify in response to changing business mandates. Time-to-market of new capabilities has also been significantly reduced in organizations that have adopted event-driven patterns [1].
- **Real-time Responsiveness:** With EDA, all information is propagated as events occur, resulting in data being delivered to other concerned destinations in real-time, rather than based on scheduled polling. The mechanism provides insights into data that are close to real-time, which is the basis of proactive and time-sensitive decision-making. Since e-commerce companies are no longer in a reactive position- where things will be done based on the things that have occurred in the past, they move towards a proactive state where activities can be undertaken simultaneously as changes take place, such as in giving customer adjustments as goods go in or out or there are swings or shifts in customer patterns. This move increases customer satisfaction and instills a competitive advantage [1].
- **Fault Tolerance:** Each of the services is scalable and fault-tolerant since they are decoupled. If a service is faulty, other components continue to run. The event routers also employ elastic buffering, which enables them to absorb spikes in demand and prevent cascading system failures [1].
- **Cost Efficiency:** The EDA-marching push model in push-based delivery would eliminate continuous polling by consumer services, thereby reducing operational costs. This impact will reduce bandwidth consumption on the network, maximize CPU utilization, shrink idle fleet capacity, and minimize the number of SSL/TLS handshakes, resulting in a decrease in infrastructure expenses [1].

C. Architectural Concepts

Successful implementation of EDA relies on understanding and applying several key architectural concepts [11],[12] :

- **Event Portal:** To design, create, discover, catalog, share, secure, and manage events and event-driven applications, specialized event portal tools are available to support these tasks. They can serve as a hub for data and joint work among architects, developers, and data scientists, and properly document and govern the event landscape.
- **Topics:** Events are categorized and have metadata called a topic, which generally consists of a tree of terms (a hierarchical text string) describing the contents of the event. To publish an event, all the publisher must know is the topic in question, and the event broker takes care of the clever distribution to all interested systems. To express interest in events, applications will subscribe to topics, which may use wildcards to match sets of similar events.
- **Event Mesh:** An event mesh is a combined and configurable guideline segment built on a system of interconnected event brokers. It serves the purpose of distributing elegant events across decoupled applications, cloud services, and devices, regardless of deployment location, whether physical or logical (in on-premises or any cloud environment, as well as IoT edge devices) [13].
- **Deferred Execution:** In EDA, the publishing of an event does not expect or wait for an instant response from consumers. The event, which remains the event broker, will continue until all interested consumers acquire it or accept it at a later time, regardless of its success. This non-blocking property can cause cascades of events, i.e., by acting on one event, subsequent events are triggered, and a condition of a chain of temporally and functionally independent but directly related operations is achieved.
- **Eventual Consistency:** Eventual consistency is a phenomenon that occurs as a direct consequence of the combination of deferred execution and asynchronous processing. The system guarantees that a particular object will eventually reach a consistent state on all systems that may be involved, even though stateful

entities may possess different states at any point in time. In the distributed systems environment, the load demands that strong consistency across components must be traded for eventual consistency, as scalability requires it. The system will require a detailed examination of consistency requirements between e-commerce components (payment authorization requires strong immediate consistency, global inventory updates can be eventually consistent), along with architectural designs of robust compensation layers to address inconsistencies in the short term and still arrive at data consistency [14], [15].

- **Command Query Responsibility Segregation (CQRS):** CQRS is a well-known design pattern used in microservices to help scale distributed architecture more efficiently. It splits the microservice responsible for carrying out the tasks that change state (commands) from the one that performs lookups (queries). Since query operations can often be orders of magnitude higher in volume than update operations, this separation allows the query service to scale independently. Event-driven architecture makes it much simpler to use CQRS. With this setup, we can have different topics carrying 'action' events, like updates or changes, which can then be picked up by several query service instances at once. Each of these services is tuned for read operations so that the system can handle parallel read requests efficiently [16].

III. HIGH-THROUGHPUT DESIGN IN EVENT-DRIVEN SYSTEMS

A. Mechanisms for High Throughput

Much of the achievement of high throughput in event-driven systems, especially in global e-commerce fulfillment, depends upon various compounding architectural and technical processes, some of which are listed below:

- **Asynchronous Communication:** High throughput is an inherent property of the fundamental principle of EDA, which is that event producers do not wait for consumers to respond. This is due to a non-blocking situation and would have led to bottlenecks in synchronous systems, resulting in a free flow of events within the system at a fast rate.
- **Parallel Processing:** Parallel processing is supported by Event brokers, like Apache Kafka and NATS. They carry this out by spreading the events across several partitions or targets, enabling multiple consumers to process the events simultaneously. This fan-out ability makes it so that individual events may spawn multiple parallel workflows but does not induce bottlenecks.
- **Partitioning Strategies:** It is essential that partitioning should be effective in capitalizing on concurrency. The system is more parallelizable into small groups, and the workload is evenly distributed, since data can be separated into several partitions across event brokers (e.g., Kafka topics). This approach avoids the use of so-called hot spots, where individual partitioning is a bottleneck due to skewness in event distribution [17].

- **Zero-Copy Techniques:** Apache Kafka and other advanced event streaming platforms rely on the zero-copy technique. They directly copy bytes of the file system to the network buffer to avoid passing through app buffers. This dramatically minimizes the number of I/O operations and CPU cycles, thereby enhancing the speed at which messages are executed and transacted [18].

B. Performance Metrics and Benchmarks

Across the board, high-throughput EDA implementations have proven capable of impressive performance. For example, in optimized mode, Apache Kafka achieves a write throughput of up to 605 MB/s with a 5 ms p99 latency at a load of 200 MB/s. Research has also revealed that 100-byte Kafka messages can be written at a rate of 57 MB/s through a single instance with acks=all and 1.28 GB/s with 32 processes and partitions [19],[20].

Using event-driven architecture in microservices can boost performance. With EDA, systems respond faster - there's about a 19.18% improvement in response times, and errors are reduced by over 34.40%. This shows that EDA helps microservices-based systems handle more requests efficiently and reduce errors. The CPU usage of EDA exceeds API-Driven Architecture by 8.52% while operating with a higher server processor. EDA provides faster performance with reduced errors yet requires additional CPU resources. The CPU time for EDA averaged 2272.97 ms, whereas API-Driven Architecture required 2079.32 ms on average [21].

Broker/Service	Throughput (Messages/sec)	Latency	Scalability Features	Message Delivery Guarantees	E-commerce Use Cases	Key Differentiators
Apache Kafka	987,000+ (150,000-200,000+)	Low (ms range)	Horizontal scaling with partitions and brokers, Consumer Groups for parallel processing	At-least-once, Exactly-once (with idempotent producers & transaction)	Real-time data pipelines, inventory updates, order processing, streaming analytics, log aggregation	High-throughput data streaming, durable log-based storage, event replay capability, robust ecosystem
RabbitMQ	High (intermediate message payloads)	Low	Vertical scaling, multiple consumers, task distribution	Reliable delivery with acknowledgments, message preservation in queues	Order processing, real-time notifications, background jobs, microservices communication	Flexible message routing (exchanges, queues, bindings), supports multiple protocols (AMQP, MQTT)
AWS Eventbridge	Millions of events/sec	Low	Serverless auto-scaling, filtering and routing rules, fan-out to multiple targets	At-least-once (with retry mechanisms)	Scalable integration, custom application events, infrastructure automation, real-time data processing	Serverless, managed event bus, extensive registry, deep AWS service integration, easy partner integration
Azure Service Bus	High (millions of messages for Event Hubs, enterprise-grade for Service Bus)	Low	Partitioning for scalability, auto-scaling (message capacity), Load balancing across competing workers	At-least-once, Exactly-once (with transaction/duplicate detection), FIFO (with sessions)	High-value enterprise messaging, order processing, financial transactions, decoupled microservices communication	Enterprise-grade messaging, robust transaction support, advanced features (session, dead-letter, auto-forwarding), hybrid cloud integration
Google Cloud Pub/Sub	500 million messages/sec (Google infra), High throughput	Low (<10 ms)	Horizontal scaling, per-message parallelism, multiple subscriptions per topic, Auto-scaling from zero	At-least-once, Exactly-once (with idempotent consumers)	Streaming analytics, data integration, parallel processing, real-time event distribution, enterprise event bus	Fully managed, global data access, seamless Kafka/Pulsar scalability with traditional message queue features

Fig. 1. Comparison of Key Event Broker Technologies for E-Commerce Fulfillment

C. Scalability Strategies

The fundamental element of high-throughput e-commerce fulfillment relies on EDA to deliver effective scalability solutions:

- **Horizontal Scaling:** The independent design of EDA makes it naturally suitable for horizontal scaling. The system maintains operational stability through independent scaling of individual services by adding more instances or partitions, which enables workload adjustments without affecting other system components. E-commerce platforms require this flexibility because their demand patterns change unpredictably and rapidly [22].
- **Elastic Buffering:** Event routers, which include Apache Kafka, Amazon SQS, and Azure Service Bus, operate as elastic buffers. The systems store high volumes of

incoming events during peak times to protect downstream systems from overload while maintaining continuous operation during demand spikes [23].

- **Load Distribution:** The design of messaging queues and event streaming platforms enables workload distribution between multiple consumers and partitions. The system achieves peak performance through this design because it prevents any single component from becoming a bottleneck and reduces processing delays.

IV. EVENT-DRIVEN ARCHITECTURES FOR E-COMMERCE FULFILLMENT

A. Application of EDA in E-Commerce Processes

Event-Driven Architecture in this sense is especially relevant to the dynamic e-commerce fulfillment, allowing the real-time automation and time-synchronization of multiple business-critical processes.

- **Order Processing:** EDA automates the entire order processing cycle. Once a customer has placed an order, a set of asynchronous activities is initiated, including the transmission of order details to the Enterprise Resource Planning (ERP) system, updating of stock details, and providing shipment date information to the customer. This facilitates real-time synchronization throughout the entire purchasing process, from placing the order to dispatch [24].
- **Inventory Management:** Good inventory management is essential to avoid over-selling or carrying excessive stocks. EDA alerts the relevant systems when stock levels are low or in excess, allowing for prompt action to be taken within the shortest time possible. This ensures that optimal levels of inventory always remain in the process to meet demand and avoid extra holding charges, as well as lost sales caused by shortages [25].
- **Payment Authorization:** EDA incorporates the payment functionalities without synchronization, so the possibly time-consuming payment processing activities would not tie down the entire order fulfillment process. Events caused by payment initiate other actions, and therefore, the customer experience is not blocked and is smooth.
- **Shipping Logistics:** As soon as an order is confirmed and payment is authorized, EDA initiates the necessary shipping processes. This involves creating shipping labels, updating fulfillment status, and communicating with third-party shipping and logistics providers. Packages can be tracked through various phases, starting with preparation and dispatching, before they finally reach their destination [26].
- **Returns Management:** EDA can connect multiple systems, including ERP, CRM, and e-commerce solutions, to make returns automatic. This also ensures the smooth running of returns, and exceptions are only handled manually, which helps create a positive customer experience even after the purchase [27].

B. Business Benefits in E-Commerce

The e-commerce business that strategically implements EDA has a variety of valuable business benefits:

- **Order Process Automation:** The processes or actions that take place after an order has been received, such as provision of data into ERP systems and stock updates, are time-consuming and mostly error-prone; automating this saves a lot of time, lowering the errors to practically zero. This automation is implemented on both the internal side, utilizing CRM software and accounting software, as well as externally, with shipping partners and fulfillment partners [28].
- **Optimized Inventory Levels:** EDA automates the alert of low stock and high stock, so that an action is taken in time. This will ensure the optimal stock holding level, which will reduce the costs of carrying excess stock as well as lost sales due to shortages [25].
- **Faster Time-to-Market:** Companies that use EDA document significant improvements in the time necessary to deliver new functions to the market. For example, in some organizations, the time-to-market of new features decreased by 67.4 percent over time due to the lack of coordination between decoupled services [26].

Benefit	Description	Key Metric/Example
Loose Coupling	Services operate independently, unaware of each other's specifics, enhancing system flexibility and resilience.	Enables independent development and deployment of services
Scalability	Allows dynamic scaling of individual components to handle fluctuating loads without impacting other parts of the system.	Organizations report 67.4% reduction in time-to-market for new capabilities
Real-time Responsiveness	Information flows as events happen, enabling immediate insights and proactive decision-making.	Reduced end-to-end latency by 73.8% compared to request-response architectures.
Fault Tolerance	Decoupled services can fail independently, with event routers buffering surges and preventing cascading failures.	Uptime improved from 99.4% to 99.7%
Cost Efficiency	Push-based event delivery reduces continuous polling, lowering network bandwidth, CPU utilization, and idle capacity.	Average reduction of 71% in infrastructure costs.
Order Process Automation	Automates actions like sending data to ERP, updating stock, and notifying customers, eliminating manual errors.	Saves time and eliminates data-entry errors for B2B order processes.
Optimized Inventory	Automates notifications for stock levels, enabling quick action and matching inventory to demand.	85% fewer overselling incidents, inventory updates propagate in <1 second.
Targeted Marketing	Provides visibility into customer behavior for personalized marketing and promotions linked to inventory.	Orchestrates services to boost search rankings and discount pricing automatically.
Accelerated Time-to-Market	Faster delivery of new features and products due to reduced coordination requirements.	67.4% reduction in time-to-market for new capabilities.
Improved Customer Experience	Enhanced performance and responsiveness contribute to a better overall customer journey.	4% increase in conversion rates due to better inventory accuracy.
Operational Efficiency	Streamlined processes and independent service development lead to more efficient operations.	23% reduction in support tickets related to inventory issues.
Increased Team Productivity	Independent development and reduced dependencies boost the productivity of development teams.	Architectural changes impacted neighboring services 91.3% less often.

Fig. 2. Benefits of Event-Driven Architectures in E-Commerce Fulfillment

- **Increased Operational Efficiency & Productivity:** Streamlining the procedures, with the capacity of creating services on their own, helps provide a higher level of operation and production within the team. This is achieved through the reduction of intra-service dependencies and, not to mention, independence among teams [26].

C. Real-World Case Studies

Leading e-commerce platforms globally have successfully implemented Event-Driven Architectures to manage their vast and complex fulfillment operations:

- **Walmart:** Walmart implements Apache Kafka as a service hub of its real-time omnichannel inventory and supply chain. Through billions of events processed daily and the event-streaming model, Walmart can optimize inventory distribution and levels, minimize cycle times, and improve accuracy across all channels (both mixed and e-commerce). To fuel this large-scale infrastructure and to assist in funneling different sources of data into a single canonical flow, the current fulfillment structure of Walmart now brings the entire system to the point of issuing timely, resilient, and elastic delivery decisions. This is accomplished within three hours, as 4 billion messages are handled during a replenishment operation. This is vital to their entire structure, both in terms of scale and precision [29].
- **eBay:** Event sourcing is utilized in eBay's continuous delivery infrastructure. All changes to the state are provided in the form of immutable event logs. This offers enhanced concurrency control (events are handled sequentially), improved observability (debugging and traceability of failures), and a more manageable codebase. Pipeline Execution Service at eBay uses the actor model (Akka) in Scala, where the flow and manageability of CI/CD pipelines are based on event sourcing. CQRS is also employed to aid read and write decoupling in terms of scale and visibility. eBay event-sourced microservices have already served millions of events per pipeline, are resilient, and can be easily maintained [30].
- **Shopify:** Shopify is a well-known organization that offers a variety of webhooks, providing e-commerce infrastructure and broadcasting real-time events such as order completion, customer activity, and status updates like delivery. Architectures of Headless commerce must enjoy the flexibility of Webhooks, as it will be possible to use APIs to easily and quickly interface with various third-party applications and synchronize with applications on multiple platforms. This allows merchants and developers to respond to customer interactions, manage their inventories more efficiently, and automate their back ends in a flexible and decentralized manner [31], [32], [33].
- **Target:** Target is a giant retail store that has an omnichannel structure, featuring both physical stores and an e-commerce outlet. Target has already deployed the use case of re-platforming their legacy retail technology

to an event-driven architecture that uses Kafka as the event streaming platform of record. The systems that Target offers, namely in-store POS systems and online retail store systems, can be decoupled and upgraded to communicate real-time, event-driven data with one another, thus supporting its omnichannel commerce initiatives, including a constant view of inventory, end-to-end ordering across channels, and replacement, among others. Such upgrades enable the maintenance of identical current product availability data, order fulfillment, and order management, including buy-online-pick-up-in-store (BOPIS), curbside pickup, shipping, dynamic replenishment, and inventory optimization [34].

V. ADVANCED PATTERNS AND TECHNOLOGIES FOR GLOBAL SCALE FULFILLMENT

A. Distributed Transaction Management

- **Saga Pattern:** The Saga pattern is a technique for managing long-running business transactions that are inherently long-lasting and particularly common in microservices architecture, where a single logical transaction involves multiple services. It breaks down a complex business transaction into several small and independent local transactions. All local transactions work atomically within one service, and after that, they begin a different service within the saga by publishing an event or message. The need to employ Saga pattern emerges due to the structural constraints that are common to traditional ACID transactions in high throughput globally dispersed e-commerce systems. Decoupled services and the ability to scale globally imply a direct need to adopt complex patterns, such as Sagas, to ensure consistency in data without compromising performance. This means that although the Sagas facilitate horizontal scaling, they also entail other complexities that must be well-designed and handled [35].
- **Choreography vs. Orchestration:** There are two main ways through which Sagas can be coordinated [36]:
 - **Choreography:** Here, the services share the events without a central controller. Each of the services (that performs local transactions) publishes the events that cause the following actions in the saga. This is a sort of orchestrated dance among services, with each service responding to the events of interest.
 - **Orchestration:** On the other hand, in the Orchestration approach, there is a central orchestrator. This specialized element processes and regulates the flow of events, ensuring the status of the entire transaction, as well as error recovery, and provides firm instructions to participant services to undertake their local transactions.

B. Data management Patterns

- **Event Sourcing (ES):** It is an architecture pattern in which all the changes done in an application state are captured as a sequence of fixed events and written to a variable event store. In a situation where the current state is all that one needs to store, Event Sourcing saves everything chronicle of the attainment of that state. This allows for reconstructing the state of any entity at any time, which is invaluable in auditing, debugging, and enforcing data consistency among distributed systems [37].

C. Global Distribution and Event Mesh

The primary role of Event Mesh in cross-border e-commerce is to synchronize data in real-time between microservices distributed across different geographical locations. This will also ensure that essential updates, such as customer profile changes, order management status changes, or billing updates, occur and spread evenly and efficiently throughout all related systems, regardless of their physical location. One possible solution to the network latency problem faced by the e-commerce sector today, as well as the data locality problem, is the concept of event mesh—a technology that dynamically routes events to any application, regardless of its deployment location. Such a capability is a direct response to the demand for seamless and real-time data exchange between different geographies and cloud infrastructures. The event mesh is a critical component in providing consistent, high-throughput e-commerce fulfillment at a global scale, across continents [26].

D. Cloud-Native Implementations

The leading cloud providers deliver an extensive suite of services capable of forming a powerful ecosystem for programming and managing high-throughput Event-Driven Architectures essential for e-commerce Order Fulfillment, which makes their adoption and usage much easier.

- **Amazon Web Services (AWS):**
 - **AWS Lambda:** Serverless compute platforms provide an execution environment, where the developers execute code that is activated by events instead of setting up and managing dedicated servers. These services are dynamically scaled to handle the demands, have built in mechanisms of fault-tolerance, and handle and process the orders in e-commerce and also update the inventories within the competitive cost structures [38].
 - **Amazon SQS (Simple Queue Service):** This service is a centralized messaging architecture that decouples the processing of messages by microservices, while continuing to act as an independent message box with equal capabilities in terms of related performance demands. It provides support for standard queues as well as first-in, first-out (FIFO) queues, and with batch processing, it can handle up to 70,000 messages per second. Other facilities, e.g., dead-letter queues,

increase reliability by ensuring the delivery of messages despite transient failures [39].

- **Amazon SNS (Simple Notification Service):** A high-throughput, low-latency, publish-subscribe messaging service best suited for fan-out to thousands or millions of endpoints and often used for real-time notifications like order confirmations or shipping notifications [40].

- **Microsoft Azure:**

- **Azure Event Grid:** A scalable, fully managed service that provides message distribution via a publish-subscribe mechanism. Can be used to build data pipelines and event-driven architectures for serverless computing, featuring high throughput and low latency, utilizing both push and pull message delivery models [41].
- **Azure Service Bus:** An enterprise message broker that provides messaging queues and publish-subscribe topics. It is intended for messages of high value, transactional guarantees, ordering (strictly First-In-First-Out, or FIFO), and duplicate handling, for the reliable management of state transitions in complex business workflows [42].

- **Google Cloud Platform (GCP):**

- **Google Cloud Pub/Sub:** A scalable messaging service that decouples producers and consumers, asynchronous messaging is typically used in streaming analytics, data integration pipelines, and for parallelizing work. They are low-latency (of the order of 100ms), high-throughput, with a throughput of 500 million messages per second through the central Google infrastructure [43].
- **Google Cloud Functions:** A serverless compute service that allows running small fragments of code in response to events originating from different GCP services. The platform facilitates easy development and employs a pay-per-use financial structure, making it an effective event-driven processing solution for e-commerce fulfillment [44].

VI. CHALLENGES AND BEST PRACTICES IN LARGE-SCALE EDA

A. Implementation Challenges

- **Schema Evolution:** The coordination of schema changes among multiple development teams and geographically decentralized locations is inherently complex. Over the lifespan of a system, one might introduce additional data fields or modify existing fields; hence, a disciplined strategy is required, such as using Avro schema in combination with a schema registry to maintain backward compatibility. It is common to ensure

that all newly added fields are optional and that reasonable default values are added to avoid alienating existing consumers [45].

- **Monitoring and Debugging Distributed Event Flows:** Following the path of business operations, tracing it through a chain of decoupled services via an Event-Driven Architecture is significantly more challenging than in monolithic applications. The asynchronous character allows one user action to trigger a series of events in numerous services with complete independence. As such, creating correlation IDs that will be carried through the entire sequence of events is necessary to relate activities across various services to each other. Consequently, effective troubleshooting and understanding the system's behavior can be achieved [46].
- **Cost Management:** The construction of an event streaming pipeline using Apache Kafka or a cloud-native messaging service can become quite costly once such platforms begin receiving much traffic. Physically carving up your data to avoid runaway costs, you must have a strategy for how to carve up your data: that is, how many partitions to create, ensuring that the right amount of data durability can be achieved within the available funds. One of the popular tricks is to allocate minimum volume subjects to only a few partitions and then impose strict retention rules to ensure that the system does not continue to store data that you do not need in reality [47].
- **Data Consistency:** One of the other large classes of headache in the EDA world is how to maintain independent, ultimately consistent services in step. It is through eventual consistency that the system can be flexible and scalable, but it also brings the opportunity for any two sections of the system to display inconsistent data. To manage this, most teams rely on existing solutions, such as Sagas, when distributed transactions are involved, and then rely on the Sagas to resolve the issues over time [35].

B. Strategies for Robustness

- **Idempotent Consumers:** Event consumers need to be designed as idempotent systems because they should produce identical results when processing the same event multiple times as they do when processing it once. The best practice of duplicate event handling is essential because distributed systems will inevitably experience duplicate event delivery due to network problems, system failures, and retry attempts. The ability to handle duplicates gracefully reduces debugging time and maintains data integrity [48].
- **Dead Letter Queues (DLQs):** For events that keep erroring out after multiple retries, send them to a Dead Letter Queue. This isolates the troublemakers, keeps the main flow running, and makes sure we don't lose any data. Once in the DLQ, we can review, fix, and re-inject them into the workflow whenever it suits [49].

- **Circuit Breakers:** The introduction of circuit breakers to a distributed system eliminates cascading failures. A circuit breaker may enable a temporary halt of calls to a failing service, giving the service time to recover without overwhelming it further or causing other services to fail, when it has started failing continually or has become slow to respond. This design ensures the fault tolerance of the architecture [26].
- **Event Replay:** A powerful debugging feature is the capability to reprocess events at any time in the event log. The ability to debug, recover bugs, or restore system state is a powerful feature. Suppose a bug corrupting data is discovered, or a new model of analytics needs to be trained to learn from historical data. In that case, event replay can be used to re-ingest the historical events and recompute them to rectify the state or to discover new insights [50].

VII. CONCLUSION

Event-driven Architectures on an enterprise scale (e.g. regionalized large-volume event processing and messaging, and large-volume secure persistent storage) are the enterprise gold standard of e-commerce fulfillment architecture. These macroeconomic events have also made the market to be volatile and intricate in its operation. These realities have come to the fore in the architectural flaws of such legacy monolithic systems. The event-sourcing paradigm, created by EDA, offers the operational flexibility, horizontal scalability, and fault isolation necessary to operate mission-critical fulfillment operations using asynchronous messaging and loosely coupled microservices.

There is a wide range of benefits associated with Event-Driven Architecture (EDA): many of them can be measured in terms of direct business outcomes. Through its natural loose coupling, services can be created and run without pre-decided interdependencies, thereby facilitating organizational agility and significantly reducing the time-to-market cycles for developing new features. The real-time dynamics of event information propagation in the architecture enable enterprises to make proactive decisions, optimize inventory, customize customer experiences, and improve operational workflows. Furthermore, the architecture-specific features contribute to fault tolerance and provide concrete cost reductions through the optimization of resource usage and the avoidance of unnecessary polling.

Although event-driven architectures have the strong potential to scale global e-commerce fulfillment, considerations should be made regarding the different complexities introduced in the implementation process due to these architectures. The coordination of schema change, resolving traceability issues on loosely coupled services, and containing infrastructure costs under high throughput conditions are not trivial problems. Maintaining the consistency of data in asynchronous systems is also difficult and requires strong patterns, such as Saga with compensating transactions, to ensure business integrity.

In the future, it is possible to identify three research lines. First, methods of adaptive routing of events are necessary that adjust the direction of events dynamically according to system load, area latency, or semantics. Second, observability in

federated event systems cannot be confined to logs and metrics anymore and instead needs to transform to deliver a cross-service, end-to-end picture, allowing root-cause analysis at a cross-cloud region and cross-organizational level. Lastly, contextual optimization using AI-driven decision-making is expected to be applied in the event streams, delivering real-time capabilities like predictive throttling, intelligent retry strategies, and customer experiences. Collectively, these opportunities of the future promised to take EDAs beyond being just reactive systems, to truly brilliant, self-optimizing fulfillment systems.

In conclusion, Event-Driven Architecture (EDA) introduces some additional architectural complexity but undoubtedly enables e-commerce fulfillment infrastructure to be scalable, distributed, and fault-tolerant. The most significant e-commerce companies confirm these architectural patterns as real-life, enterprise-level solutions, not just abstract concepts. Therefore, in organizations striving to gain a competitive edge in the global market, both the adoption and optimization of event-driven architectures are a mission-critical obligation towards sustainable scalability, efficient operations, and differentiated customer experiences.

Disclaimer: This document reflects the views of the individual author(s) and does not represent Walmart's current or future stance.

REFERENCES

- [1] L. Lazzari and K. Farias, "Uncovering the Hidden Potential of Event-Driven Architecture: A Research Agenda," Aug. 10, 2023, arXiv: arXiv:2308.05270. doi: 10.48550/arXiv.2308.05270.
- [2] F. Rademacher, J. Sorgalla, and S. Sachweh, "Challenges of Domain-Driven Microservice Design: A Model-Driven Perspective," *IEEE Softw.*, vol. 35, no. 3, pp. 36–43, May 2018, doi: 10.1109/MS.2018.2141028.
- [3] F. Aulkeimeier, M. A. Paramartha, M.-E. Iacob, and J. Van Hillegersberg, "A pluggable service platform architecture for e-commerce," *Inf. Syst. E-Bus. Manag.*, vol. 14, no. 3, pp. 469–489, Aug. 2016, doi: 10.1007/s10257-015-0291-6.
- [4] W. Hasselbring and G. Steinacker, "Microservice Architectures for Scalability, Agility and Reliability in E-Commerce," in 2017 IEEE International Conference on Software Architecture Workshops (ICSAW), Gothenburg, Sweden: IEEE, Apr. 2017, pp. 243–246. doi: 10.1109/ICSAW.2017.11.
- [5] S. Dinesh and Y. MuniRaju, "SCALABILITY OF E-COMMERCE IN THE COVID-19 ERA," *Int. J. Res. -GRANTHAALAYAH*, vol. 9, no. 1, pp. 123–128, Jan. 2021, doi: 10.29121/granthaalayah.v9.i1.2021.3032.
- [6] Y. Che, M. Yuan, Y. Zhang, and L. Zhao, "Cross-border E-commerce and China's Exports during the COVID-19 Pandemic," *China World Econ.*, vol. 32, no. 3, pp. 215–242, May 2024, doi: 10.1111/cwe.12537.
- [7] V. Cristea, F. Pop, C. Dobre, and A. Costan, "Distributed Architectures for Event-Based Systems," in Reasoning in Event-Based Distributed Systems, vol. 347, S. Helmer, A. Poulouvasilis, and F. Xhafa, Eds., in Studies in Computational Intelligence, vol. 347., Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 11–45. doi: 10.1007/978-3-642-19724-6_2.
- [8] P. Raj, S. Vanga, and A. Chaudhary, *Cloud-Native Computing: How to Design, Develop, and Secure Microservices and Event-Driven Applications*, 1st ed. Wiley, 2022. doi: 10.1002/9781119814795.
- [9] R. Tsyganok, "Methodology for Building Scalable Microservice Architectures on Go for High-Load E-Commerce Platforms," *Univers. Libr. Eng. Technol.*, vol. 01, no. 02, pp. 42–46, 2024, doi: 10.70315/uloap.ulete.2024.0102007.
- [10] Y. Huang, Y. Chai, Y. Liu, and J. Shen, "Architecture of next-generation e-commerce platform," *Tsinghua Sci. Technol.*, vol. 24, no. 1, pp. 18–29, Feb. 2019, doi: 10.26599/TST.2018.9010067.
- [11] "https://solace.com/what-is-event-driven-architecture/."
- [12] "https://dev.to/jimjunior/a-practical-introduction-to-the-event-driven-architecture-dol."
- [13] "https://solace.com/wp/architects-guide-to-implementing-event-driven-architecture/".
- [14] E. Sakic and W. Kellerer, "Impact of Adaptive Consistency on Distributed SDN Applications: An Empirical Study," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 12, pp. 2702–2715, Dec. 2018, doi: 10.1109/JSAC.2018.2871309.
- [15] M. Diogo, B. Cabral, and J. Bernardino, "Consistency Models of NoSQL Databases," *Future Internet*, vol. 11, no. 2, p. 43, Feb. 2019, doi: 10.3390/fi11020043.
- [16] A. Debski, B. Szczepanik, M. Malawski, S. Spahr, and D. Muthig, "A Scalable, Reactive Architecture for Cloud Applications," *IEEE Softw.*, vol. 35, no. 2, pp. 62–71, Mar. 2018, doi: 10.1109/MS.2017.265095722.
- [17] T. P. Raptis, C. Cicconetti, and A. Passarella, "Efficient topic partitioning of Apache Kafka for high-reliability real-time data streaming applications," *Future Gener. Comput. Syst.*, vol. 154, pp. 173–188, May 2024, doi: 10.1016/j.future.2023.12.028.
- [18] J. Park, C. Min, H. Yeom, and Y. Son, "Design and Implementation of Efficient and Transparent Zero Copy Read," *IEEE Access*, vol. 12, pp. 174078–174093, 2024, doi: 10.1109/ACCESS.2024.3502688.
- [19] "https://www.confluent.io/blog/kafka-fastest-messaging-system/".
- [20] "https://developer.confluent.io/learn/kafka-performance/."
- [21] A. Rahmatulloh, F. Nugraha, R. Gunawan, and I. Darmawan, "Event-Driven Architecture to Improve Performance and Scalability in Microservices-Based Systems," in 2022 International Conference Advancement in Data Science, E-learning and Information Systems (ICADEIS), Bandung, Indonesia: IEEE, Nov. 2022, pp. 01–06. doi: 10.1109/ICADEIS56544.2022.10037390.
- [22] A. Kwan, J. Wong, H.-A. Jacobsen, and V. Muthusamy, "HyScale: Hybrid and Network Scaling of Dockerized Microservices in Cloud Data Centres," in 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS), Dallas, TX, USA: IEEE, July 2019, pp. 80–90. doi: 10.1109/ICDCS.2019.00017.
- [23] H. Wu, Z. Shang, and K. Wolter, "Performance Prediction for the Apache Kafka Messaging System," in 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), Zhangjiajie, China: IEEE, Aug. 2019, pp. 154–161. doi: 10.1109/HPCC/SmartCity/DSS.2019.00036.
- [24] M. Gördesli and A. Varol, "Comparing Interservice Communications of Microservices for E-Commerce Industry," in 2022 10th International Symposium on Digital Forensics and Security (ISDFS), Istanbul, Turkey: IEEE, June 2022, pp. 1–4. doi: 10.1109/ISDFS55398.2022.9800784.
- [25] Srikanth Yerra, "Enhancing Inventory Management through Real-Time Power BI Dashboards and KPI Tracking," *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol.*, vol. 11, no. 2, pp. 944–951, Mar. 2025, doi: 10.32628/CSEIT25112458.
- [26] Venkata Anil Kumar Nilisetty, "Event-Driven Microservices for Autonomous Supply Chain Coordination: A Technical Deep Dive," *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol.*, vol. 11, no. 2, pp. 218–226, Mar. 2025, doi: 10.32628/CSEIT25112355.
- [27] "https://solace.com/blog/business-case-eda-retail/".
- [28] "https://www.netsuite.com/portal/resource/articles/erp/automate-order-processing.shtml".
- [29] "https://www.confluent.io/blog/walmart-real-time-inventory-management-using-kafka/".
- [30] "https://innovation.ebayinc.com/stories/event-sourcing-in-action-with-ebays-continuous-delivery-team/".
- [31] "https://shopify.dev/docs/apps/build/webhooks".
- [32] "https://www.kurrent.io/event-sourcing".
- [33] "https://www.shopify.com/enterprise/blog/headless-commerce".
- [34] "https://www.confluent.io/kafka-summit-nyc17/simplifying-omni-channel-retail-at-scale/".

- [35] M. De Heus, K. Psarakis, M. Fragkoulis, and A. Katsifodimos, "Distributed transactions on serverless stateful functions," in *Proceedings of the 15th ACM International Conference on Distributed and Event-based Systems, Virtual Event Italy: ACM*, June 2021, pp. 31–42. doi: 10.1145/3465480.3466920.
- [36] C. K. Rudrabhatla, "Comparison of Event Choreography and Orchestration Techniques in Microservice Architecture," *Int. J. Adv. Comput. Sci. Appl.*, vol. 9, no. 8, 2018, doi: 10.14569/IJACSA.2018.090804.
- [37] M. Overeem, M. Spoor, S. Jansen, and S. Brinkkemper, "An Empirical Characterization of Event Sourced Systems and Their Schema Evolution -- Lessons from Industry," *J. Syst. Softw.*, vol. 178, p. 110970, Aug. 2021, doi: 10.1016/j.jss.2021.110970.
- [38] W.-T. Lin et al., "Tracking Causal Order in AWS Lambda Applications," in *2018 IEEE International Conference on Cloud Engineering (IC2E)*, Orlando, FL: IEEE, Apr. 2018, pp. 50–60. doi: 10.1109/IC2E.2018.00027.
- [39] "<https://aws.amazon.com/sqs/features/>".
- [40] "<https://aws.amazon.com/sns/features/>".
- [41] "<https://learn.microsoft.com/en-us/azure/event-grid/>".
- [42] "<https://azure.microsoft.com/en-us/products/service-bus/>".
- [43] "<https://www.bigindustries.be/blog/exploring-messaging-and-streaming-technologies-part5-google-pub/sub>".
- [44] "<https://cloud.google.com/functions/docs>".
- [45] Lviv Polytechnic National University, E. Maltsev, and O. Muliarevych, "Beyond JSON: Evaluating Serialization Formats for Space-Efficient Communication," *Adv. Cyber-Phys. Syst.*, vol. 9, no. 1, pp. 9–15, May 2024, doi: 10.23939/acps2024.01.009.
- [46] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 2762–2780, July 2023, doi: 10.1109/TDSC.2022.3162857.
- [47] T. P. Raptis, C. Cicconetti, and A. Passarella, "Efficient topic partitioning of Apache Kafka for high-reliability real-time data streaming applications," *Future Gener. Comput. Syst.*, vol. 154, pp. 173–188, May 2024, doi: 10.1016/j.future.2023.12.028.
- [48] G. Wang et al., "Consistency and Completeness: Rethinking Distributed Stream Processing in Apache Kafka," in *Proceedings of the 2021 International Conference on Management of Data, Virtual Event China: ACM*, June 2021, pp. 2602–2613. doi: 10.1145/3448016.3457556.
- [49] "<https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-dead-letter-queues>".
- [50] P. Alvaro and A. Quinn, "Deterministic Record-and-Replay: Zeroing in only on the nondeterministic actions of the process," *Queue*, vol. 22, no. 4, pp. 120–129, Aug. 2024, doi: 10.1145/3688088.